

CAN-FD <-> SPI Bridge

Mainline Zephyr on the Arduino UNO Q

Architecture, implementation, testing & performance

STM32U585 (Cortex-M33, Zephyr 4.4) . QCM2290 (Qualcomm Linux)

Executive summary

This report documents a working, bidirectional **CAN-FD to SPI bridge** built on **mainline Zephyr 4.4** running on the Arduino UNO Q's STM32U585 microcontroller, paired with the board's QCM2290 applications processor running Qualcomm Linux. The MCU runs its CAN-FD controller in on-chip loopback and exchanges frames with the Linux side over SPI, with the STM32 as SPI slave and Linux as master via `/dev/spidev0.0`. A hardware ready (RDY) line provides lossless flow control.

The solution was built incrementally and measured on real hardware at every step: from a classic-CAN baseline, CAN-FD (64-byte payloads) was added, then per-block CRC-16 integrity and sequence-gap detection, then an optimization round. The optimized bridge sustains **1,040 round-trip frames/s and 66.5 kB/s of CAN payload** — an **11.1× payload-throughput improvement** over the classic-CAN baseline — while adding CAN-FD, integrity checking, and flow control.

Headline results

Metric	Baseline	Final (optimized)	Gain
Round-trip frame rate	750 frames/s	1,040 frames/s	1.4x
CAN payload throughput	6.0 kB/s	66.5 kB/s	11.1x
Max payload per frame	8 bytes	64 bytes (CAN-FD)	8x
Integrity checking	none	CRC-16 per block	added
Flow control	none	RDY line, lossless	added

All numbers are medians of five timed runs on the physical board; methodology in the Benchmarking section.

1. System architecture

The bridge spans two processors on a single board. The **QCM2290** (Qualcomm applications processor, Yocto-based Qualcomm Linux) is the SPI **master** and the system's northbound interface. The **STM32U585** (Cortex-M33) runs mainline Zephyr, owns the CAN-FD peripheral, and acts as SPI **slave**. They are wired by the board's SPI3 bus plus a dedicated ready/handshake line.

CAN-FD <-> SPI bridge - system architecture

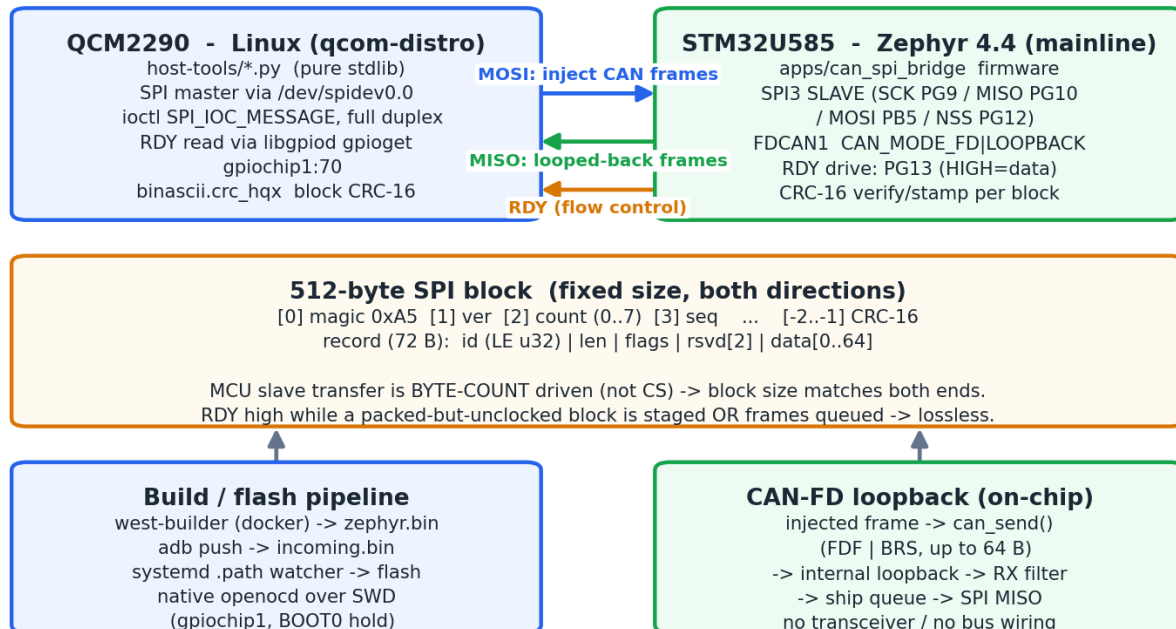


Figure 1 — End-to-end architecture: build/flash pipeline, the SPI block protocol, CAN-FD loopback, and the three signal paths (MOSI inject, MISO return, RDY).

1.1 MCU side (STM32U585, Zephyr 4.4)

- **FDCAN1** via Zephyr's CAN API in CAN_MODE_FD | CAN_MODE_LOOPBACK with bitrate switching (BRS). Enabled by a device-tree overlay (kernel clock PLL1_Q, pins PD0/PD1). Up to 64 data bytes/frame.
- **SPI3 as slave** on board-default pins (SCK PG9, MISO PG10, MOSI PB5, NSS PG12). The QCM master supplies clock + chip-select; the driver sets NSS to hardware-input.
- **RDY line** on PG13 (= QCM gpiochip1:70), driven HIGH whenever unread frames are staged or queued for the master — basis of lossless flow control.
- **CRC-16** (CCITT-FALSE) in firmware over every block, to stamp outbound and verify/reject corrupt inbound command blocks.
- Two threads: a CAN-RX thread (loopback frames into a ship queue, raises RDY) and the SPI thread (per transfer: parse-and-inject inbound, pack-and-stamp outbound).

1.2 Linux side (QCM2290, Qualcomm Linux)

- **SPI master** via /dev/spidev0.0 — a purpose-built DT node (compatible = "arduino,unoq-mcu") on the GENI SE. Full-duplex transfers via SPI_IOC_MESSAGE.
- **RDY read** via libgpiod (gpioget) on gpiochip1:70 so the master clocks transfers only when data is ready.
- **Host tooling** is pure Python stdlib (the device interpreter ships without spidev / ctypes / 3rd-party modules). Block CRC uses C-implemented binascii.crc_hqx, matching the firmware bit-for-bit.

1.3 The SPI block protocol

SPI has no message framing and — critically — the STM32 slave transfer is **byte-count driven, not chip-select driven**: deasserting NSS does not end a transfer mid-buffer. The protocol uses a **fixed-size block** both ends agree on exactly, same layout both directions:

```
[0] magic 0xA5 [1] version [2] count (0..7) [3] seq ... [-2..-1] CRC-16 (LE)
```

```
record (72 B): id (LE u32) | len (0..64) | flags | rsvd[2] | data[0..64]
```

- **MOSI** (Linux->MCU): command block; MCU injects each record as a CAN-FD frame.
- **MISO** (MCU->Linux): up to 7 looped-back CAN frames per 512-byte block.
- **RDY** stays HIGH while a packed-but-unclocked block is staged *or* frames remain queued, so the master never stops one read early — this makes the link lossless under burst.

2. Implementation & toolchain

The whole build-and-flash loop runs from a laptop over a single ADB connection (the device is not on the network). It was engineered to be fast and panic-free — an early attempt to run the flasher in a container kernel-panicked the 1.7 GB device during image decompression, so the flasher runs natively.

2.1 Build -> flash pipeline

- **west-builder** — Docker-wrapped `west build` (Zephyr SDK 1.0.1, bind-mounted workspace) compiles any app for `arduino_uno_q`, emitting `zephyr.bin`.
- **ADB push** drops the binary on the device; a **systemd path-unit watcher** notices the change and runs the flasher automatically.
- **Native openocd** flashes the STM32 over SWD (bit-banged via `gpiochip1`), holding `BOOT0` low across reset so the MCU boots the new image rather than the ROM bootloader.

2.2 Verification without a serial console

The board's Zephyr console UART does not surface on the QCM serial path, so correctness was proven by **SWD-readable RAM counters**: the firmware exposes globals (`g_frames_injected`, `g_can_rx`, `g_frames_packed`, `g_rx_crc_err`, ...) read live with `openocd mdw`. This gives an independent, ground-truth cross-check of every Linux-side measurement.

3. What works

Every capability below was demonstrated on the physical board, not in simulation.

Capability	Status	Evidence
Build / flash / boot pipeline	Working	<code>west build -> adb -> watcher -> SWD flash -> run</code>
SPI-slave data path	Working	full-duplex echo verified byte-exact
FDCAN on-chip loopback	Working	<code>TX=RX=match</code> counters equal via SWD
CAN -> SPI -> Linux (RX)	Working	frames decoded on <code>/dev/spidev0.0</code>
Linux -> MCU CAN inject (TX)	Working	MOSI command blocks -> <code>can_send()</code>
RDY flow control (lossless)	Working	200 frames > queue depth, 0 lost
CAN-FD 64-byte / BRS	Working	64-byte payloads round-trip byte-exact
Block CRC-16 integrity	Working	5/5 corrupt rejected, 15/15 good pass
Sequence-gap detection	Working	host tracks seq continuity per block
Real CAN bus (transceiver)	Not tested	loopback only; no transceiver wired

4. Testing methodology

Each feature was validated by a dedicated test checking an observable, falsifiable outcome — and, wherever possible, cross-checked against the MCU's own SWD counters so the Linux and firmware views must agree.

4.1 Correctness tests

- **SPI-slave echo**: slave preloads a known banner (A0..BF) and echoes each received block on the next transfer; master verified the banner, then that block N+1 returned exactly what block N sent.
- **FDCAN loopback**: firmware counted transmitted, received and content-matched frames; all three equal (via SWD), and the program counter confirmed execution from flash.
- **Bidirectional + RDY losslessness**: 200 frames injected through a ship queue of depth 64 (loss would occur without flow control). Result: 200 injected, 200 read back, **0 missing, 0 unexpected**, cross-checked by equal MCU counters.
- **CAN-FD payload**: 12 frames with full 64-byte payloads injected and read back byte-for-byte identical.
- **CRC integrity (negative test)**: five valid plus five deliberately CRC-corrupted blocks sent. MCU injected exactly the 15 valid frames and incremented its CRC-error counter exactly 5 times — corrupt blocks rejected, never acted on.

4.2 Why these tests are trustworthy

- **Dual-sided accounting**: host counts what it sent/received; MCU independently counts via SWD. A bug on either side shows up as a mismatch.
- **Falsifiable thresholds**: tests assert exact equality (0 missing / 0 extra / counters equal), not "looks plausible".
- **Integrity verified on the wire**: the CRC check is exercised with intentionally corrupt input, proving it actually rejects bad data.

5. Benchmark methodology

Throughput was measured by a harness that drives the bridge as hard as possible and times a fixed amount of work, kept deliberately simple and repeatable so before/after numbers are comparable.

- **Metric**: sustained round-trip throughput — each transfer injects frames (MOSI) and reads looped-back frames (MISO). Reported as frames/s and CAN-payload kB/s.
- **Warm-up**: ~300 frames of untimed transfers first, to fill the pipeline.
- **Timed window**: back-to-back full-duplex transfers until a target frame count is read back, measured with `time.monotonic()`.
- **No artificial sleeps**, and in the throughput loop no per-transfer `gpioget` (a subprocess fork per transfer would dominate). Flow-control correctness is proven separately by the RDY-gated loss test.
- **Repetition**: five runs; the **median** is reported. Runs were tightly clustered (typically <1% spread), indicating a stable measurement.
- **Integrity during benchmarking**: the harness verifies the CRC of every returned block and tracks sequence continuity. In all reported runs `crc_err = 0`.

One honest caveat: the throughput loop saturates the link and samples returned blocks rather than draining every one under RDY gating, so its sequence counter shows occasional expected gaps (the host isn't reading every block the MCU produces). That is a property of the max-rate sampler, not data loss — losslessness is proven independently by the RDY-gated test in section 4.1.

6. Performance results

The bridge was benchmarked at four stages: the classic-CAN baseline, after adding CAN-FD, after adding CRC + sequence-gap detection, and after the optimization round.

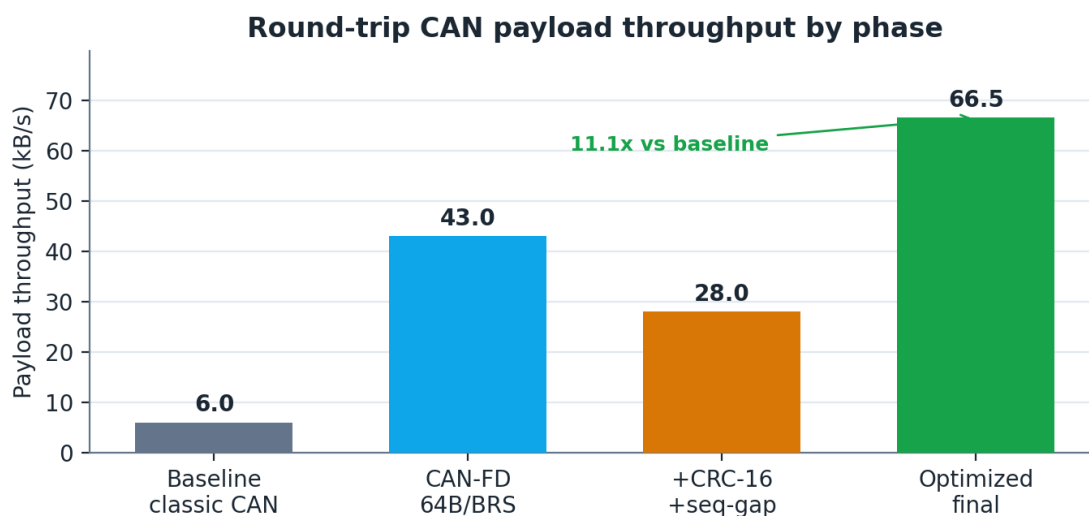


Figure 2 — CAN payload throughput by phase. CAN-FD delivers the largest single jump (8x data/frame); the optimization round more than doubles it again to 66.5 kB/s.

Phase	Payload	SPI	frames/s	kB/s	vs base
Baseline (classic CAN)	8 B	1 MHz	750	6.0	-
CAN-FD (64 B / BRS)	64 B	1 MHz	671	43.0	+7.2x
+ CRC-16 + seq-gap	64 B	1 MHz	438	28.0	+4.7x
Optimized (final)	64 B	2 MHz	1,040	66.5	+11.1x

Table values are medians of five runs. "vs base" compares payload kB/s to the classic-CAN baseline.

6.1 CAN-FD: payload over frame rate

Enabling CAN-FD slightly lowers raw frame rate (750->671 frames/s) because each frame rides in a larger SPI block, but it carries 8x the data, so effective payload throughput rises 7.2x. For any real CAN workload, payload throughput is the metric that matters.

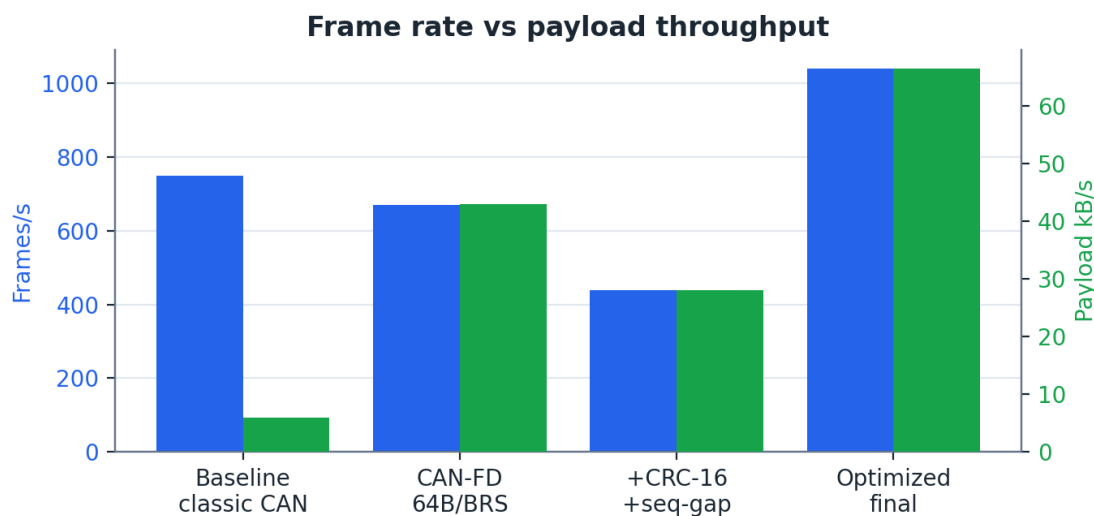


Figure 3 — Frame rate (blue) versus payload throughput (green).

7. Optimization analysis

Adding CRC initially cost 35% of throughput — but profiling showed the cost was entirely on the host, in a pure-Python CRC loop, not the (cheap) firmware CRC. The optimization round addressed three bottlenecks in turn:

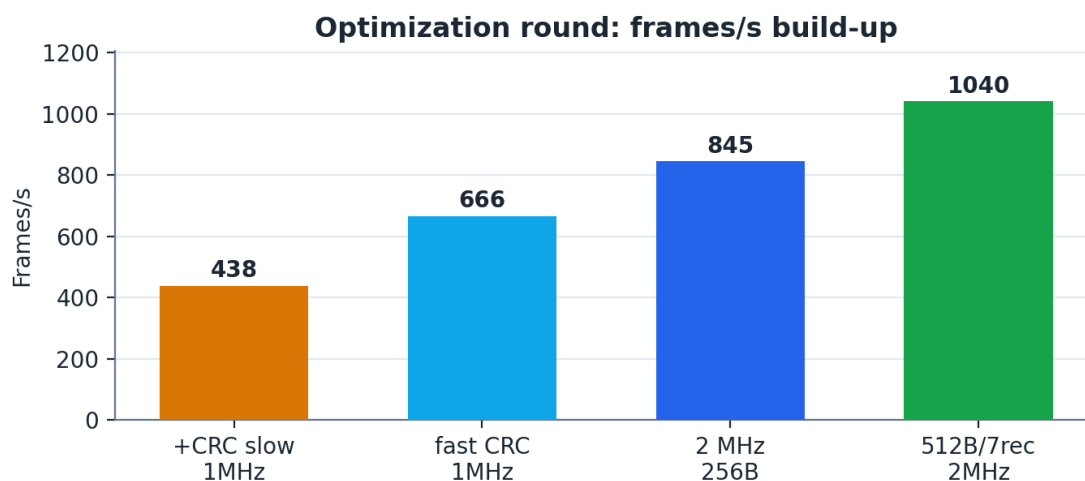


Figure 4 — Frame-rate build-up across the three optimizations, from the un-optimized +CRC low point.

- **Fast CRC** — replacing the pure-Python CRC-16 with C-implemented `binascii.crc_hqx` (verified to match the firmware bit-for-bit) recovered almost the entire CRC penalty: 438->666 frames/s (+52%).
- **Higher SPI clock** — raising the bus from 1 to 2 MHz lifted throughput to 845 frames/s (+27%). A clock sweep established the safe ceiling.
- **Larger blocks** — moving from 256-byte/3-frame to 512-byte/7-frame blocks amortizes per-transfer ioctl, CRC and slave-CPU overhead across more frames, reaching the final 1,040 frames/s (+23%).

7.1 The SPI clock ceiling

The STM32 SPI-slave driver runs in interrupt mode (no DMA). A clock sweep showed the slave keeps up cleanly to ~3 MHz but collapses at 4 MHz and above — the ISR can no longer keep the SPI FIFO fed between per-block CPU work, and returned blocks become invalid. The CRC check correctly flags this rather than silently passing corrupt data. 2 MHz was selected as the reliable operating point.

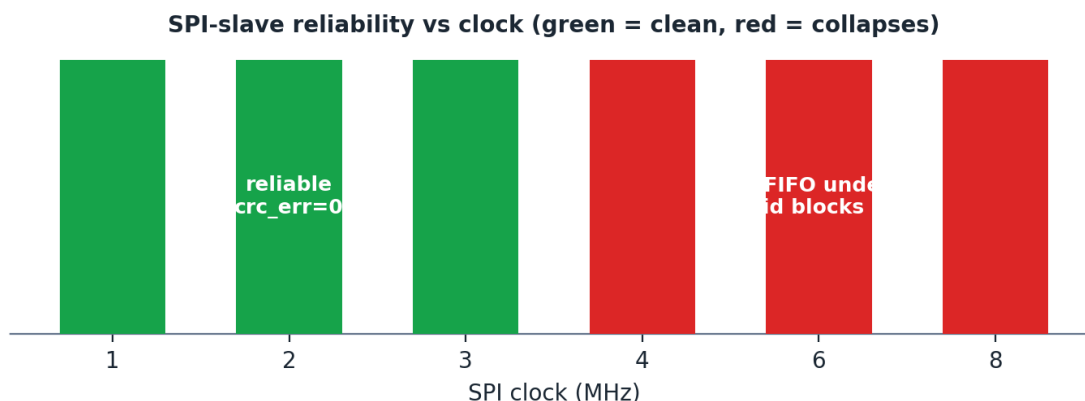


Figure 5 — SPI-slave reliability versus clock. Green = clean (CRC errors = 0); red = slave underruns, no valid blocks. A DMA slave path is the route to higher clocks.

8. Limitations & future work

- **Loopback only:** all CAN traffic is on-chip loopback. A real bus needs a CAN transceiver on PD0/PD1 and a second node; the firmware path is unchanged.
- **SPI clock capped at ~3 MHz** by the interrupt-mode slave driver. Enabling the STM32 SPI DMA slave path is the most promising route to higher throughput.
- **Test ID coverage:** standard IDs and a range of payload sizes are covered; extended IDs and error/RTR frames are not yet.
- **Host tooling is reference-grade Python:** a production northbound service (e.g. a Go SPI daemon exposing SocketCAN or a network API) would replace the benchmark scripts.
- **No back-pressure on inject:** the Linux->MCU direction assumes the MCU TX mailbox keeps up; a busy real bus would need TX flow control too.

Appendix — key source artifacts

Firmware: `apps/can_spi_bridge/` (Zephyr app + overlay). Host tools: `host-tools/` (`bench.py`, `can_spi_bridge.py`, `can_spi_fd_test.py`, `crc_test.py`). Build/flash: `west-builder/`, `openocd-native/`, `watcher/`. Raw benchmark log: `bench-results/results.md`.